

[Add-ons \(/categories/add-ons\)](/categories/add-ons) / [All Add-ons \(/categories/add-on-documentation\)](/categories/add-on-documentation) / [Redis® Cloud \(/articles/rediscloud\)](/articles/rediscloud)

Redis® Cloud

☰ Table of Contents [expand]

🕒 Last updated April 08, 2026



This add-on is operated by Redis

From the creators of Redis®. Enterprise-Class Redis for Developers (w/Free plan)

Redis Cloud (<https://redis.io/cloud/>) is a fully managed cloud service for hosting and running your Redis database. You can quickly and easily get your apps up and running with Redis Cloud through its add-on for Heroku - tell us how much memory you need and get started fast with your first Redis database. You can then add more Redis databases and increase or decrease the memory size of your plan without affecting your existing data.

Redis Cloud offers true high availability with its in-memory dataset replication and instant auto-failover mechanism, combined with its fast storage engine. You can easily import an existing Redis dataset to any of your Redis Cloud databases. For paid plans, we back up your data every 24 hours, and you can back up your data manually at any time.

Get started

Select Install Redis Cloud (<https://dashboard.heroku.com/provision-addon?addonServiceId=d0e95e8c-a963-4437-9cca-d351a1407f2a&planId=ffea926b-cb2d-412d-9a75-0ca73b95ea61>) to install the Redis Cloud add-on.

You can also use the Heroku command-line interface to install Redis Cloud. Run the following `heroku` command to install the add-on and create a Free 30MB database.

```
$ heroku addons:create rediscloud
```



You can also specify the plan size by adding the plan size in MB. For example, run the following command to install the add-on and create a 1GB database.

```
$ heroku addons:create rediscloud:1000
```



A list of all available plans can be found [here \(https://elements.heroku.com/addons/rediscloud\)](https://elements.heroku.com/addons/rediscloud).

After Redis Cloud has been added, you can find a `REDISCLLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>) in your `heroku config` containing the username, password, hostname and port of your first Redis Cloud database.

```
$ heroku config:get REDISCLLOUD_URL
http://[username]:[password]@[hostname]:[port]
```



Connect to your database

You can connect to your database with any Redis client.

The following sections show how to connect with these official, supported, and maintained client libraries:

- Java with `jedis`
- Python with `redis-py`
- Node.js with `node-redis`

See the Redis docs (<https://redis.io/docs/latest/develop/connect/clients/>) to learn how to connect with the other official client libraries.

You can also connect with these libraries and frameworks:

- Ruby with `redis-rb`
- PHP with `Predis`
- Django with `django-redis-cache`

Connect with Java

`jedis` (<https://redis.io/docs/latest/develop/connect/clients/java/jedis/>) is an easy to use Redis Java client. You can download the latest build from github (<http://github.com/xetorthio/jedis/downloads>) or use it as a maven dependency:

```
<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
  <version>2.0.0</version>
  <type>jar</type>
  <scope>compile</scope>
</dependency>
```

Configure connection to your Redis Cloud service using `REDISCLLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>) and the following code snippet:

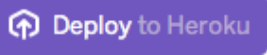
```
try {
    URI redisUri = new URI(System.getenv("REDISCLLOUD_URL"));
    JedisPool pool = new JedisPool(new JedisPoolConfig(),
        redisUri.getHost(),
        redisUri.getPort(),
        Protocol.DEFAULT_TIMEOUT,
        redisUri.getUserInfo().split(":",2)[1]);
} catch (URISyntaxException e) {
    // URI couldn't be parsed.
}
```

Testing (Java)

```
Jedis jedis = pool.getResource();
jedis.set("foo", "bar");
String value = jedis.get("foo");
// return the instance to the pool when you're done
pool.returnResource(jedis);
```



A Java sample application, running on **Play!** (<https://www.playframework.com>), is available at GitHub.

Browse the **source code** (<https://github.com/RedisLabs/rediscloud-java-sample>) or  **Deploy to Heroku** (<https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-java-sample>)

Connect with Python

`redis-py` (<https://redis.io/docs/latest/develop/connect/clients/python/>) is the official client to connect to Redis from Python.

Use pip to install it:

```
sudo pip install redis
```

Configure connection to your Redis Cloud service using the `REDISCLLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>) and the following code snippet:

```
import os
import urlparse
import redis
url = urlparse.urlparse(os.environ.get('REDISCLLOUD_URL'))
r = redis.Redis(host=url.hostname, port=url.port, password=url.password)
```

Testing (Python):

```
>>> r.set('foo', 'bar')
True
>>> r.get('foo')
'bar'
```

Connect with Node.js

`node_redis` (<https://github.com/redis/node-redis>) is a complete Redis client for Node.js.

You can install it with:

```
npm install redis
```

Configure connection to your Redis Cloud service using `REDISCLLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>) and the following code snippet:

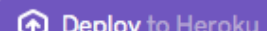
```
var redis = require('redis');
var client = redis.createClient(process.env.REDISCLLOUD_URL, {no_ready_check: true});
```

Testing (Node.js)

```
client.set('foo', 'bar');
client.get('foo', function (err, reply) {
  console.log(reply.toString()); // Will print `bar`
});
```



A Node.js sample application is available at GitHub.

Browse the **source code** (<https://github.com/RedisLabs/rediscloud-node-sample>) or  **Deploy to Heroku** (<https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-node-sample>)

heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-node-sample)

Connect with Ruby

`redis-rb` (<https://github.com/redis/redis-rb>) is a very stable and mature redis client and the easiest way to access Redis from Ruby.

Install `redis-rb` :

```
sudo gem install redis
```

Configure Redis on Rails

For Rails 2.3.3 up to Rails 3.0, update the `config/environment.rb` to include the redis gem:

```
config.gem 'redis'
```

For Rails 3.0 and above, update the Gemfile:

```
gem 'redis'
```

And then install the gem with Bundler:

```
bundle install
```

Lastly, create a new `redis.rb` initializer in `config/initializers/` and add the following code snippet. For Rails before version 4:

```
if ENV["REDIS_CLOUD_URL"]
  uri = URI.parse(ENV["REDIS_CLOUD_URL"])
  $redis = Redis.new(:host => uri.host, :port => uri.port, :password => uri.password)
end
```

For Rails 4:

```
if ENV["REDIS_CLOUD_URL"]
  $redis = Redis.new(:url => ENV["REDIS_CLOUD_URL"])
end
```

Configure Redis on Sinatra

Add this code snippet to your configure block:

```
configure do
  . . .
  require 'redis'
  uri = URI.parse(ENV["REDIS_CLOUD_URL"])
  $redis = Redis.new(:host => uri.host, :port => uri.port, :password => uri.password)
  . . .
end
```

Use Redis on Unicorn

No special setup is required when using Redis Cloud with a Unicorn server. Users running Rails apps on Unicorn should follow the instructions in the Configure Redis from Rails section and users running Sinatra apps on Unicorn should follow the instructions in the

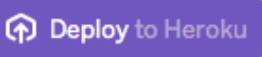
Configure Redis on Sinatra section.

Testing (Ruby)

```
$redis.set("foo", "bar")
# => "OK"
$redis.get("foo")
# => "bar"
```



A Sinatra sample application is available at GitHub.

Browse the **source code** (<https://github.com/RedisLabs/rediscloud-ruby-sinatra-sample>) or  (<https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-ruby-sinatra-sample>)

Connect with PHP

Predis (<https://github.com/nrk/predis>) is a flexible and feature-complete PHP client library for Redis.

Instructions for installing the Predis (<https://github.com/nrk/predis>) library can be found here (<https://github.com/nrk/predis#how-to-use-predis>).

Loading the library to your project should be straightforward:

```
<?php
// prepend a base path if Predis is not present in your "include_path".
require 'Predis/Autoloader.php';
Predis\Autoloader::register();
```

Configure connection to your Redis Cloud service using `REDISCLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>) and the following code snippet:

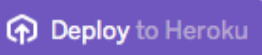
```
$redis = new Predis\Client(array(
    'host' => parse_url($_ENV['REDISCLLOUD_URL'], PHP_URL_HOST),
    'port' => parse_url($_ENV['REDISCLLOUD_URL'], PHP_URL_PORT),
    'password' => parse_url($_ENV['REDISCLLOUD_URL'], PHP_URL_PASS),
));
```

Testing (PHP)

```
$redis->set('foo', 'bar');
$value = $redis->get('foo');
```



A PHP sample application is available at GitHub.

Browse the **source code** (<https://github.com/RedisLabs/rediscloud-php-sample>) or  (<https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-php-sample>)

Configure Redis on Django

Redis can be used as the back-end cache for Django.

To do so, install django-redis-cache (<https://pypi.python.org/pypi/django-redis-cache>):

```
pip install django-redis-cache
```

Next, configure your `CACHES` in the `settings.py` file:

```
import os
import urlparse
redis_url = urlparse.urlparse(os.environ.get('REDIS_CLOUD_URL'))
CACHES = {
    'default': {
        'BACKEND': 'redis_cache.RedisCache',
        'LOCATION': '%s:%s' % (redis_url.hostname, redis_url.port),
        'OPTIONS': {
            'PASSWORD': redis_url.password,
            'DB': 0,
        }
    }
}
```

Testing (Django)

```
from django.core.cache import cache
cache.set("foo", "bar")
print cache.get("foo")
```



A Django sample application is available at GitHub.

Browse the [source code \(https://github.com/RedisLabs/rediscloud-django-sample\)](https://github.com/RedisLabs/rediscloud-django-sample) or [Deploy to Heroku \(https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-django-sample\)](https://heroku.com/deploy?template=https://github.com/RedisLabs/rediscloud-django-sample)

Open Redis Cloud console

The Redis Cloud console lets you make changes to your database settings, add extra databases to your plan, and view metrics for your databases.

To access the Redis Cloud console, run:

```
heroku addons:open rediscloud
```

Alternatively, open the Redis Cloud add-on from your application's dashboard at heroku.com.



Anybody on your team that has **App permissions** (<https://devcenter.heroku.com/articles/app-permissions>) greater than View can make changes to your database configuration on the Redis Cloud console as if they were an Owner on Redis Cloud. See **Team management roles** (<https://redis.io/docs/latest/operate/rc/security/access-control/access-management/#team-management-roles>) on the Redis Cloud docs for more information.

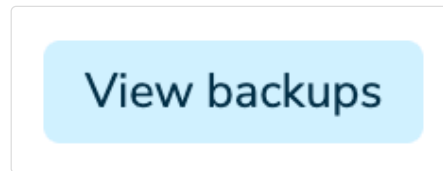
From the Redis Cloud console, you can:

- Back up your data and view backups
- Add more Redis databases to your plan
- View database metrics

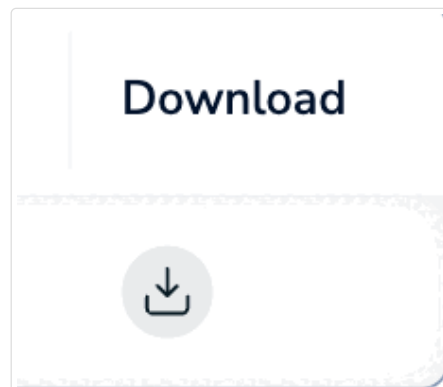
- Import data from an existing Redis server

Back up your data

By default, Redis stores your backups to a default backup repository. To view your backups from the Redis Cloud console, select your database from the database list. In the **Configuration** tab, under **Durability**, select **View backups**.

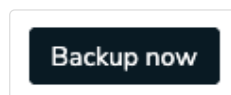


Select **Download** to download a backup file.



The default repository only retains daily backups for one week. To learn how to change your backup location, see [Back up a database](https://redis.io/docs/latest/operate/rc/databases/back-up-data/) (Redis Cloud docs) (<https://redis.io/docs/latest/operate/rc/databases/back-up-data/>).

Select **Backup now** to back up your database manually.



Backups are available only for paid plans.

Add Redis databases to your plan

For paid plans, Redis Cloud allows you to add multiple Redis databases without interfering with your other databases from the Redis Cloud console.

Each plan has a limit for the amount of databases you can create. After you reach the database limit or the memory limit for your plan, you won't be able to add any more databases.

Your first Redis database is created automatically upon launching the Redis Cloud add-on and its URL and credentials are maintained in `REDISCLLOUD_URL` config vars (<https://devcenter.heroku.com/articles/config-vars>).

To add more databases from the Redis Cloud console:

1. Select **Subscriptions** from the menu to see your plan with all of its databases.

2. Select **+ Create database in this subscription**.

+ Create database in this subscription

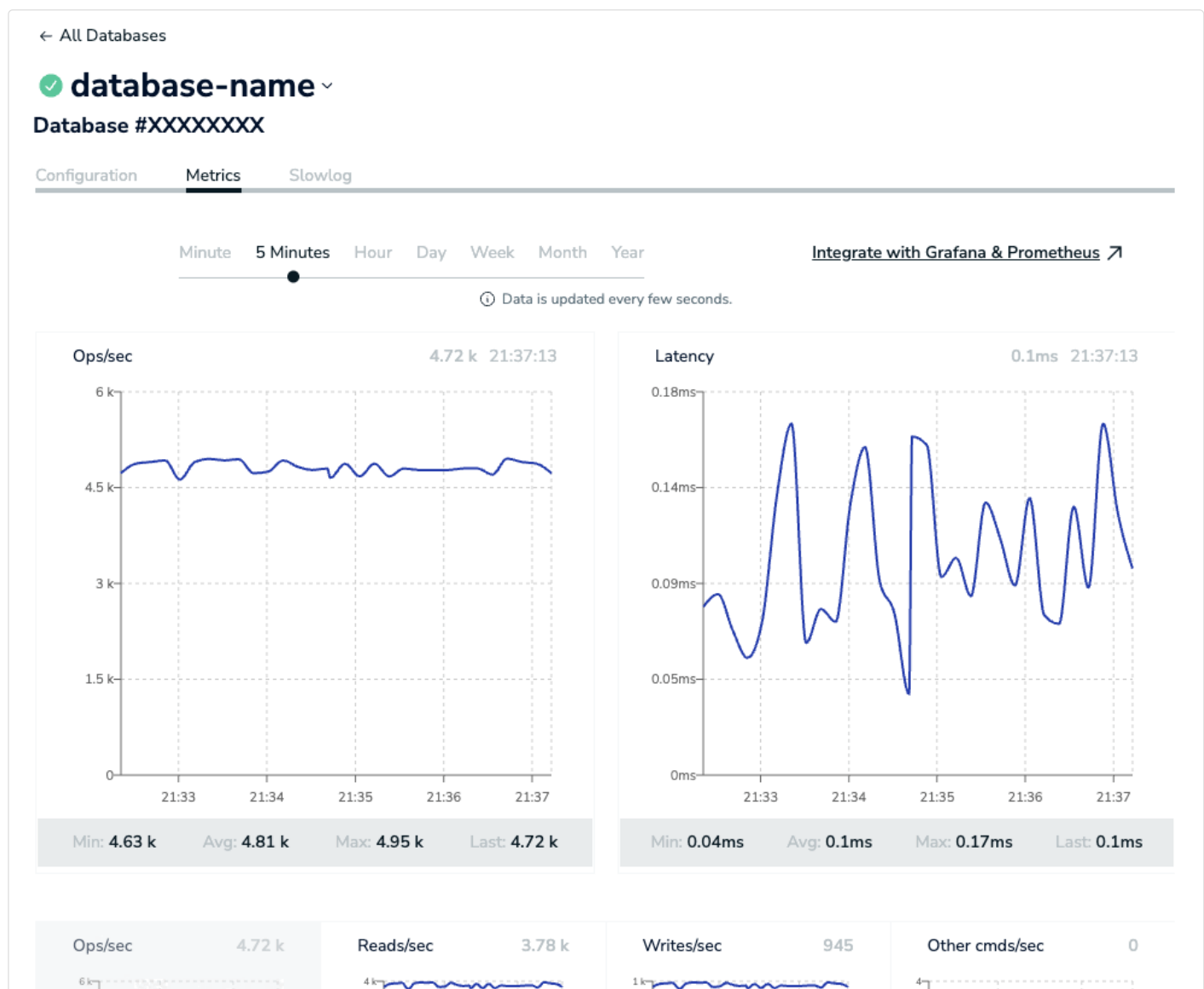
3. Enter the database name and select your new database's settings. For more info on these settings, see the Redis Cloud docs (<https://redis.io/docs/latest/operate/rc/databases/create-database/create-pro-database-existing/#general-section>).
4. Select **Activate database** to activate your new database.

After your new database is ready, select the **Connect** button to learn how to connect to it.

Connect

View database metrics

To view metrics from the Redis Cloud console, select your database from the database list, and then select the **Metrics** tab.





For more information on the metrics available in the Redis Cloud console, see [Monitor database performance \(Redis Cloud docs\)](https://redis.io/docs/latest/operate/rc/databases/monitor-performance/) (<https://redis.io/docs/latest/operate/rc/databases/monitor-performance/>).

Import data from an existing Redis server

You can create a Redis Cloud instance with the following command:

```
$ heroku addons:create rediscloud:<plan size> --app <app_name>
```

You can directly import your data into your Redis Cloud instance by opening the Redis Cloud console and following these steps (<https://redis.io/docs/latest/operate/rc/databases/import-data/#redis-server>).

Alternatively, you can migrate your data from your Heroku Data for Redis database to your Redis Cloud Heroku add-on database by using RIOT (<https://redis.io/docs/latest/integrate/riot/>).



If migrating your data from Heroku Data for Redis, get your Heroku Data for Redis credentials from your config vars in the **dashboard** (<https://devcenter.heroku.com/articles/config-vars#using-the-heroku-dashboard>) or by running the **redis:credentials CLI command** (<https://devcenter.heroku.com/articles/managing-heroku-redis-using-cli#redis-credentials>).

Upgrade plan

Upgrading your plan is easy and instant. It requires no code change and has no effect on your existing datasets. You can use the 'heroku addons:upgrade' command to upgrade to a new plan.

```
$ heroku addons:upgrade rediscloud:Free --app <app_name>
```

```
$ heroku addons:upgrade rediscloud:[size_in_mb]
```

For example, run this command to upgrade to a 5 GB database:

```
$ heroku addons:upgrade rediscloud:5000
```

Remove the add-on

To remove the Redis Cloud add-on, run:



This will destroy all data and cannot be reversed.

```
$ heroku addons:destroy rediscloud
```

Support

All Redis Cloud support and runtime issues should be submitted via the Heroku Support channels (<https://devcenter.heroku.com/articles/support-channels>). We recommend also reaching out to Redis support (<https://redis.io/support/>) for urgent issues. We are also available on twitter @Redisinc (<https://twitter.com/Redisinc>).

Additional resources

- Redis Cloud documentation (<https://redis.io/docs/latest/operate/rc/>)
- Develop with Redis docs (<https://redis.io/docs/latest/develop/>)
- Redis Client docs (<https://redis.io/docs/latest/develop/connect/clients/>)